CortexAI — AI-Powered Local CCTV Safety Monitor

Track: AI for Public Safety

Problem Statement

Traditional CCTV monitoring systems have three core weaknesses:

1. **Alarm fatigue** — passive, motion-based surveillance triggers alerts on any pixel change, with no understanding of *what* is actually happening. This floods operators with false alarms until real emergencies get ignored.
2. **Cloud dependency** — most "smart" camera systems send video to the cloud for analysis, introducing latency and serious privacy risk, since footage of people's homes or workplaces leaves the premises.
3. **Black-box detection** — pattern-matching models flag anomalies without explaining *why*, making it impossible to verify an alert or use it as evidence after the fact.

Our project addresses all three by replacing pixel-based motion detection with a **local, reasoning-capable AI model** that watches the camera feed, judges severity like a human would, explains its judgment in plain language, and only alerts when something actually warrants attention.

Our Approach

Instead of analyzing every single video frame continuously (which is computationally expensive and unnecessary for spotting real emergencies), our system works in short, regular cycles:

1. **Capture** — Every 10 seconds, the camera records a short clip.
2. **Sample** — A handful of frames (8) are evenly picked from that clip — enough to understand what's happening without processing every frame.
3. **Reason** — These frames are sent to a **Gemma vision-language model running entirely on the local machine** (via Ollama). The model doesn't just detect motion — it looks at the scene and reasons about it, the same way a human security guard would glance at a monitor.
4. **Classify** — The model returns a structured judgment:
   - **Severity**: none / low / medium / high
   - **Event type**: e.g. normal activity, fall, fight, intrusion, fire
   - **Reasoning**: a short, plain-language explanation of what it saw and why
   - **Confidence**: how sure the model is

5. **Log** — Every single judgment, even "nothing happening," is saved to an evidence log with a timestamp. This creates an auditable trail that a human reviewer or investigator can actually check — solving the "black box" problem directly.
6. **Alert** — If severity crosses a threshold (e.g. "medium" or above), the system sends an automatic WhatsApp alert containing the event type, confidence, and reasoning. For high-severity events, it can also place an automated voice call that reads out the incident summary.
7. **Cooldown** — To prevent alarm fatigue, the system won't send repeat alerts for the same ongoing incident within a set time window (e.g. 2 minutes) — it alerts once, clearly, rather than spamming.

Why This Design Solves the Stated Problems

Problem	Our Solution
Alarm fatigue	We don't alert on motion — we alert on a *reasoned* severity judgment, with a cooldown so one ongoing event doesn't trigger repeated alerts.
Cloud latency & privacy risk	The AI model runs completely on the local device. No video frame is ever sent over the internet — only a short text alert is sent, and only if something is actually detected.
Black-box reasoning	Every judgment, including "nothing happened," is logged with a plain-language explanation. This gives a human-readable, timestamped audit trail instead of an unexplained alarm.

System Architecture

```
Camera (webcam / CCTV feed)
        |
        v
Capture 10-second clip every cycle
        |
        v
Sample 8 representative frames
        |
```

```
        v
Local Gemma vision model (via Ollama, runs on-device)
        |
        v
Structured verdict: severity, event type, reasoning, confidence
        |
   -----------------------------
   |                           |
Evidence log                severity above threshold?
(forensic/audit trail)              |
                              cooldown check
                                     |
                    WhatsApp alert (+ voice call for high severity)
```

Technology Used

- **Vision-Language Model**: Gemma (running locally via Ollama) — chosen specifically because it runs on-device, which is what makes the privacy/latency solution possible.
- **Computer Vision / Capture**: OpenCV for webcam access and frame sampling.
- **Alerting**: WhatsApp messaging and voice calling for real-time notification to a responsible person.
- **Evidence Logging**: A structured, timestamped log of every model judgment, kept for review and verification.

Current Status

This is a working proof-of-concept, built and tested as a laptop-camera demo standing in for a real CCTV deployment.
"The capture-to-reasoning pipeline is fully
working end-to-end. WhatsApp alerting is in process.

What Makes This Different From Typical Anomaly Detection

Most CCTV "AI" systems today are pattern-matchers: they compare pixels to a baseline and flag deviation, with no understanding of context. Our system instead asks a reasoning model to actually *look* at the scene and describe, in its own words, what's happening and why it matters — closer to how a human observer would work, but running locally and continuously. This is what lets us give a real explanation for every alert instead of just a beep.

Future Improvements

- **Adaptive monitoring frequency**: check more frequently when a low-level concern is detected, and less frequently when the scene is calm, to balance responsiveness with efficiency.
- **Two-step confirmation**: re-check with fresh frames before firing a high-severity alert, to further reduce false positives.
- **Multi-camera support**: extend the same pipeline to multiple camera feeds in a facility, all logging to one central evidence trail.
- **Real CCTV/RTSP integration**: replace the laptop webcam with an actual CCTV camera feed for a production deployment.

Conclusion

Our project shows that a small, locally-run reasoning model can replace traditional motion-based surveillance with something meaningfully smarter: a system that watches, thinks, explains itself, and only speaks up when it actually matters — all without a single frame of video ever leaving the device.